Computing GC content with for loops

BILD 62

Quick check in...



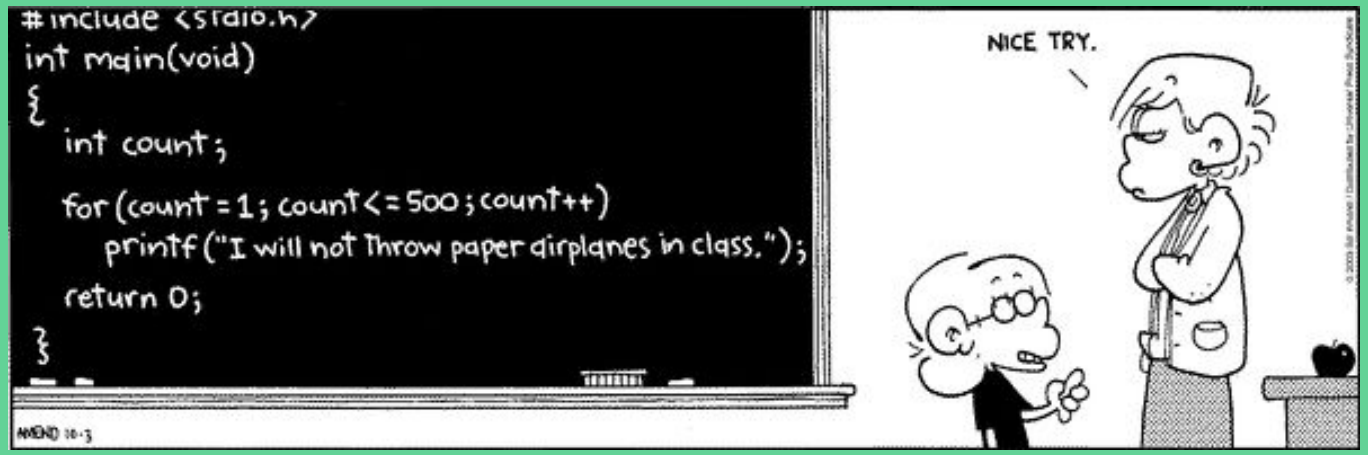
https://www.menti.com/blhpa9c4jpki

Recap of last time: computing GC content with conditionals

- Can we do this with `elif` statements?
- Can we alert the user if the function gets incorrect input (a string of incorrect length)?

A **loop** is a procedure to repeat a piece of code.
(another way of saying this is that it **iterates** through code)

- Loops enable you to re-run blocks of code for as many times as you need.
- Python has two main ways to run loops: **`for & while`**

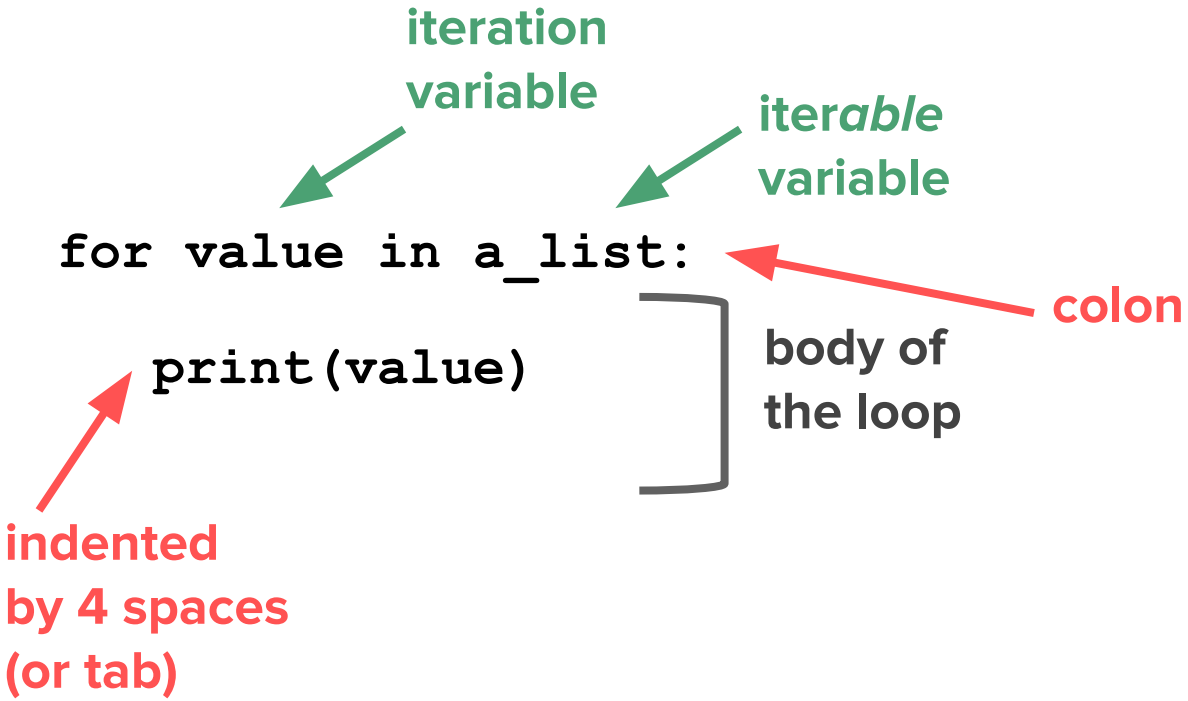


Bill Amend, FoxTrot, October 3, 2003

Objectives for this week

- Write a **`for`** loop to iterate over elements in an object
- Use a **counter** in a loop
- Use a **`for`** loop to count GC content and CCAAT boxes in a DNA string
- Write **`while`** loops and implement **`continue`** and **`break`**
- Develop a **growth mindset** towards your programming growth

`for` loop syntax



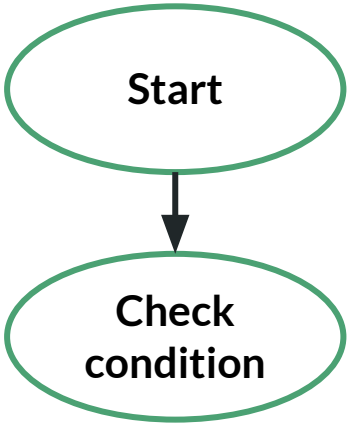
A **for loop** is a procedure to repeat code for every element in a sequence.

`for` loop syntax

```
a_list = [1,2,3]

for value in a_list:

    print(value)
```



`for` loop syntax

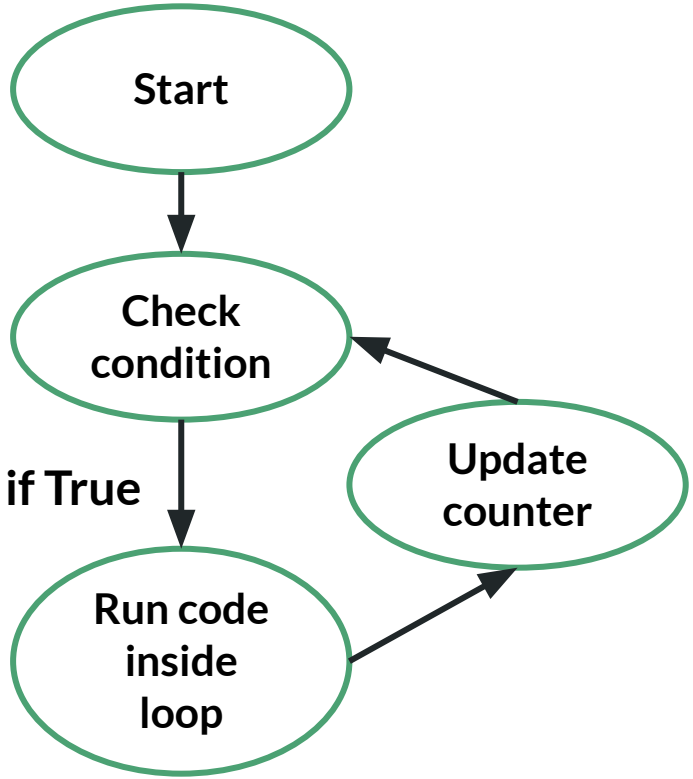
```
a_list = [1,2,3]

for value in a_list:

    print(value)
```

```
1
```

output



`for` loop syntax

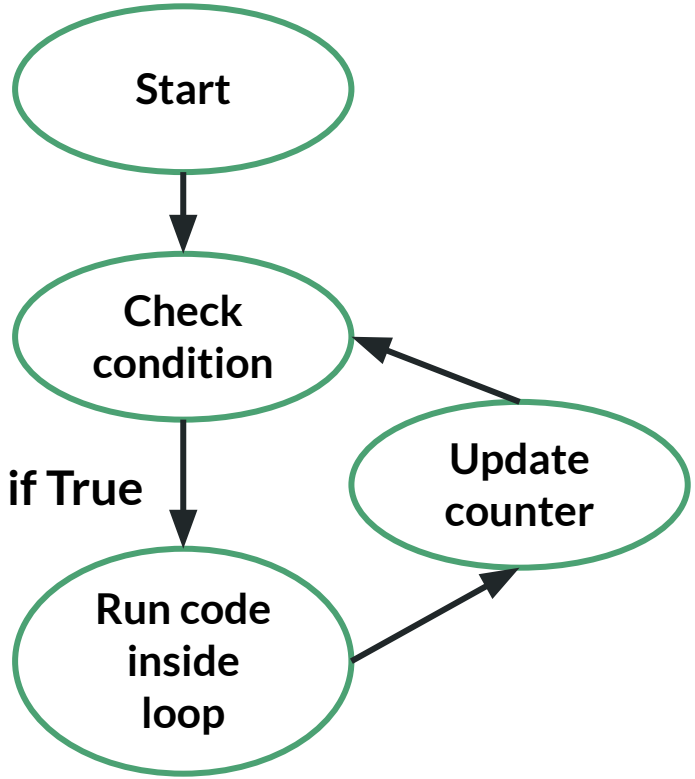
```
a_list = [1,2,3]

for value in a_list:

    print(value)
```

```
1
2
```

output



`for` loop syntax

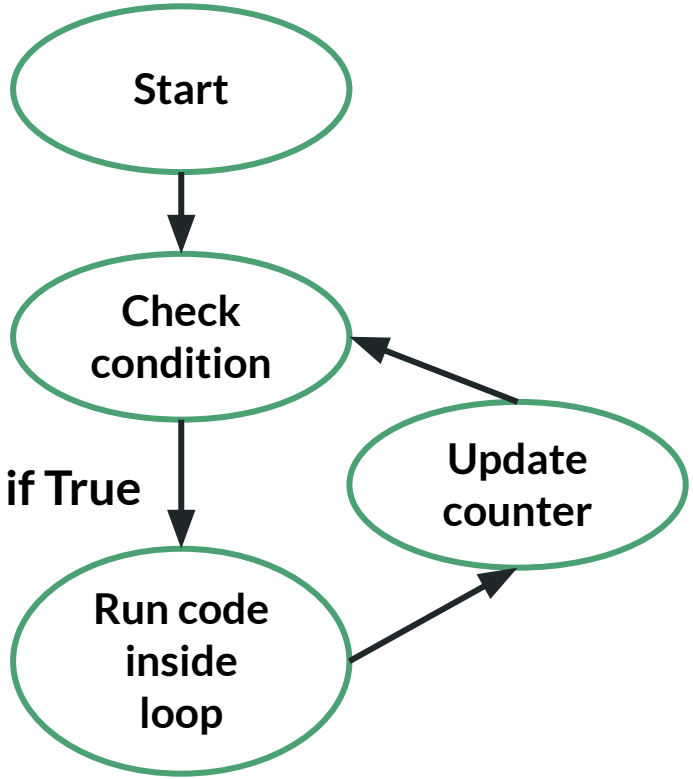
```
a_list = [1,2,3]

for value in a_list:

    print(value)
```

```
1
2
3
```

output



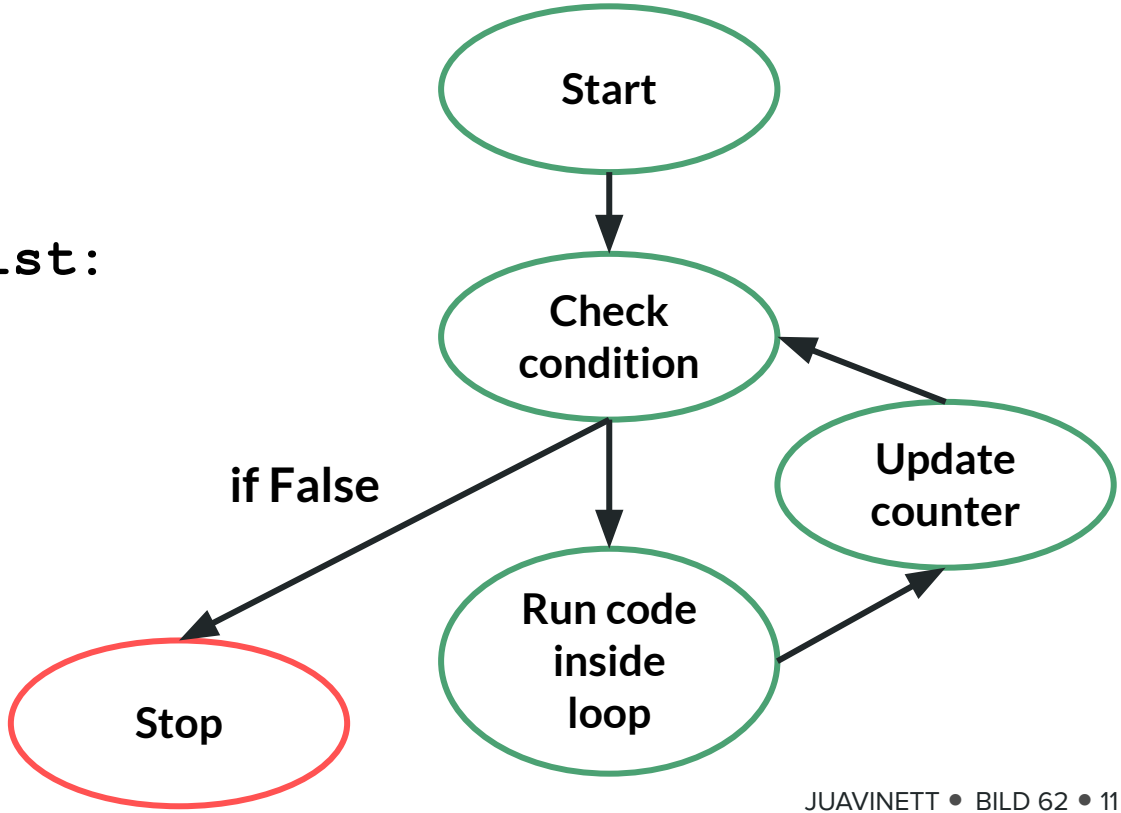
`for` loop syntax

```
a_list = [1,2,3]

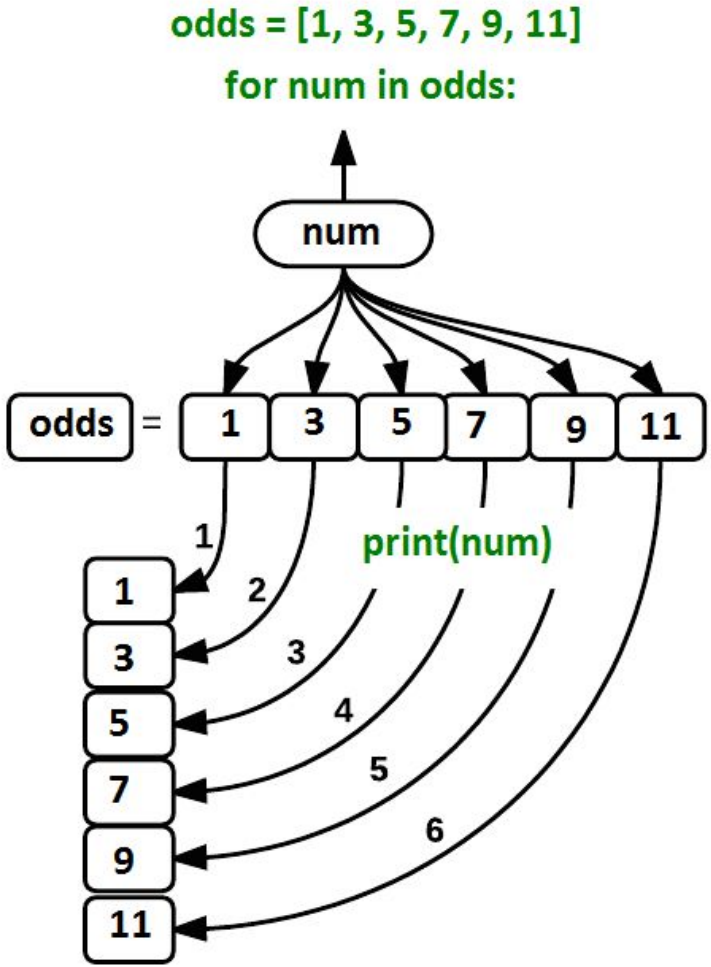
for value in a_list:

    print(value)
```

```
1
2
3
```
output



Another way to visualize working through a loop
(source)



efficiency benefit of `for` loops

Each of these would accomplish the same thing:

Option #1: 2+ lines of code

```
for value in a_list:

    print(value)
```

Option #2: as many lines of code as there are list entries

```
print(a_list[0])
print(a_list[1])
print(a_list[2])
...
```

Second task: count the # of "CAT" boxes (`CCAAT`) in a string of DNA.



The "CAT" box generally appears near the spot where transcription begins!

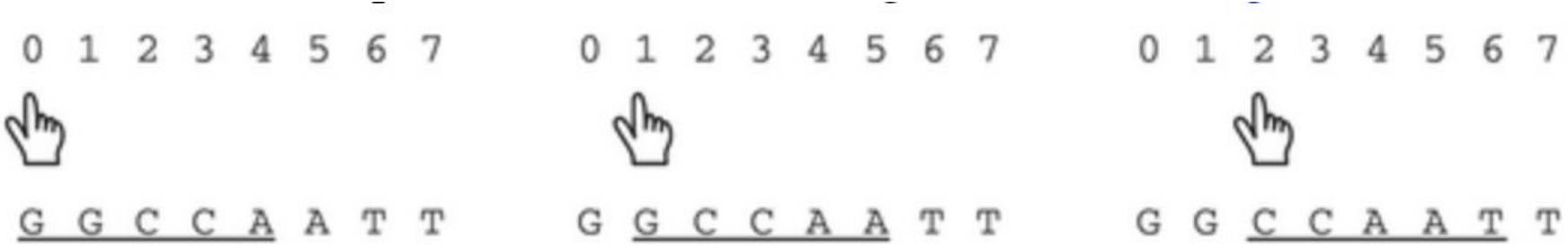
Proteins that bind here seem important for learning and memory (e.g. Arguello et al., 2013)

```
>>> countCCAAT('GGCCAATTGCCAAT')
>>> 2
```

We can also loop over a list of indices!

Let's say we want to look for a "CAT" box, a common motif in DNA, with the sequence `CCAATT`

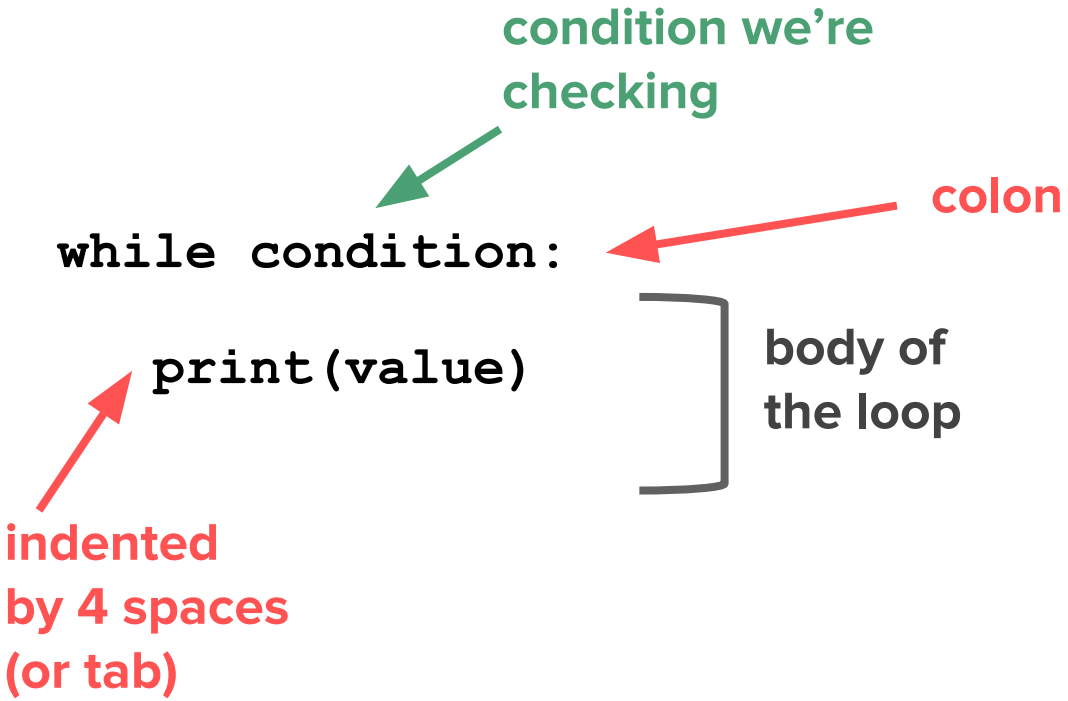
Since we want to look at a **slice** of DNA, rather than looping through individual items in the string, we need the indices.



Into the notebook...

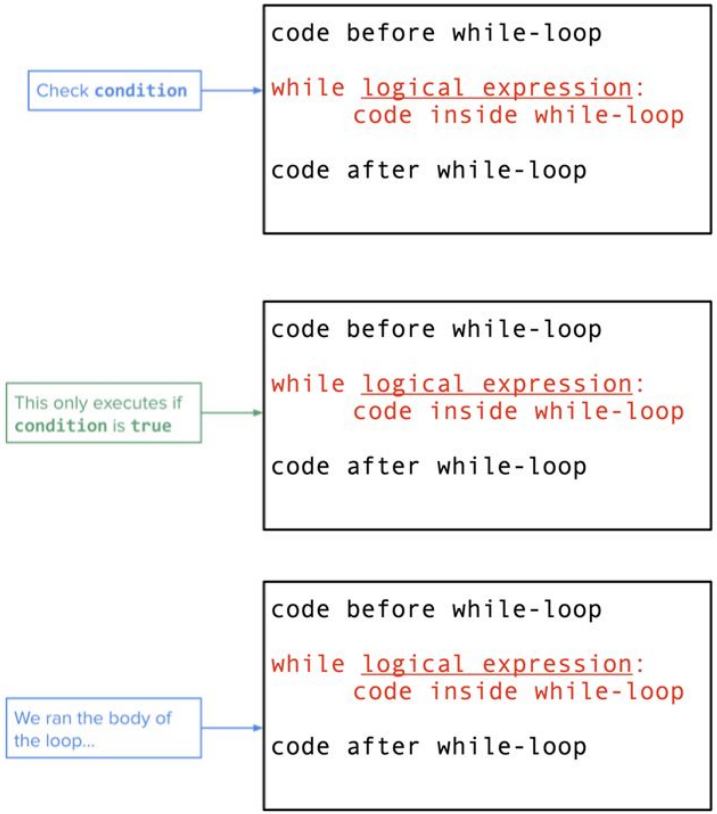


`while` loop syntax



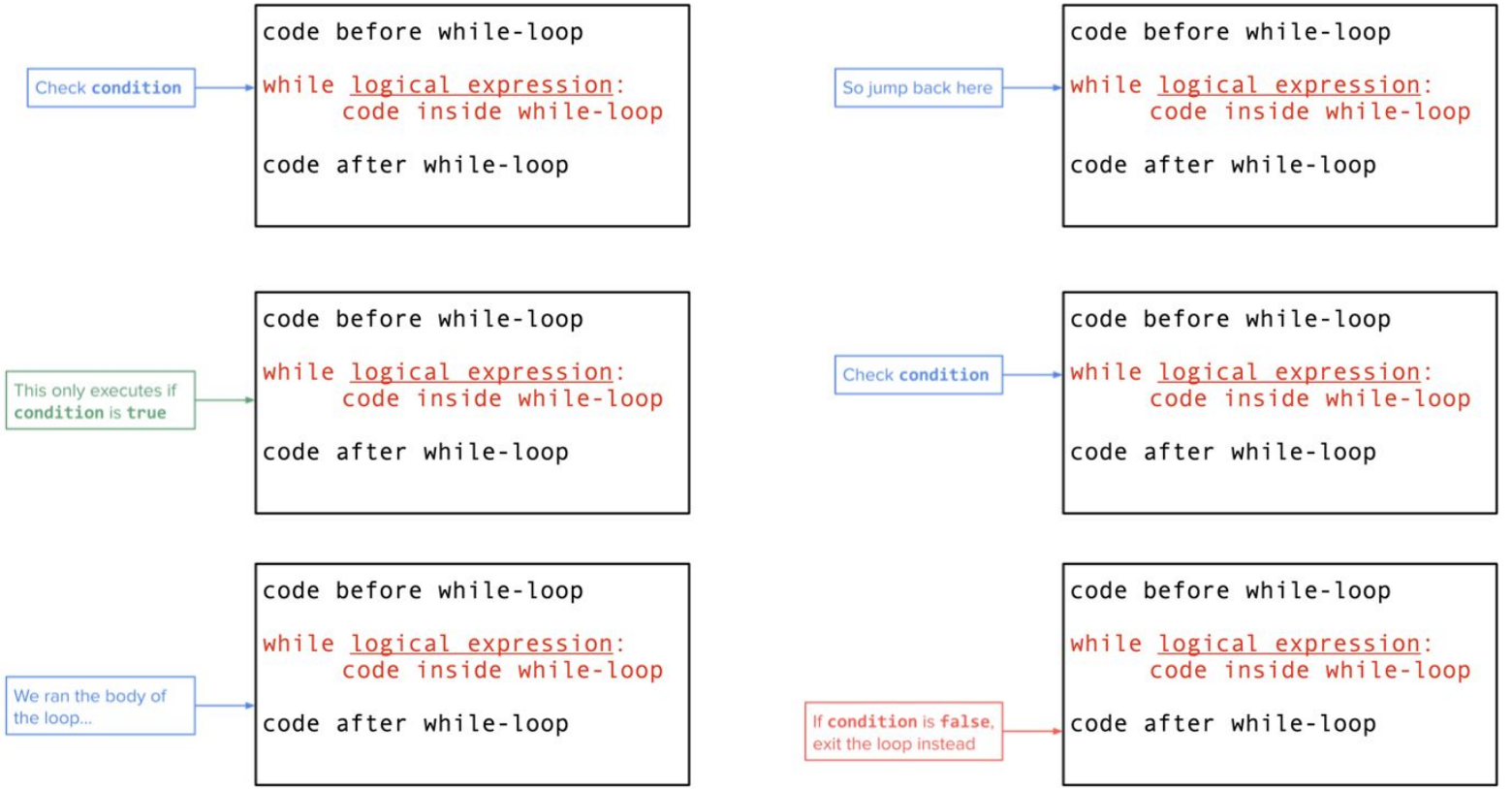
While this condition is true, the loop will run!

It will repeat until the condition is no longer True.



Order of execution in a while loop

(Image credit: Niema Moshiri & Sabeel Mansuri)



Order of execution in a while loop

(Image credit: Niema Moshiri & Sabeel Mansuri)

Topics from this lecture & corresponding notebook

- Syntax of **for** and **while** loops
- How to iterate through strings, lists, and dictionaries
- Using a counter to count loop iterations
- Looping over lists of indices
- Calling functions within functions
- Using **`break`** to interrupt a loop, and **`continue`** to skip a loop
- Functions we learned: **`range()`, `enumerate()`**

Resources

Stepik Introduction to Python book, Chapter 3

Whirlwind Tour of Python: Control Flow